TRƯỜNG ĐẠI HỌC SÀI GÒN

Khoa Công nghệ thông tin

ĐỀ ÁN CUỐI KÌ

BÀI TOÁN PHÂN LỚP TIỀN THẬT VÀ TIỀN GIẢ TRÊN DATASET BANKNOTE AUTHENTICATION

Danh sách thành viên		
STT	**MSSV**	**Họ tên**
1	3124411042	Lê Quốc Bình
2	3124411076	Nguyễn Minh Hải
3	3124411136	Phạm Nguyên Khôi
4	3124411293	Phan Nguyễn Trường Thịnh
5	3124411213	Đào Thiện Phát

Thành phố Hồ Chí Minh, tháng 3 năm 2026

Bảng phân công

STT	MSSV	Họ tên	Công việc
1	3124411042	Lê Quốc Bình	Viết report, làm slide, viết code phần Analysis, Tuning
2	3124411076	Nguyễn Minh Hải	Viết code, đánh giá & nhận xét phần Processing, Evaluation
3	3124411136	Phạm Nguyên Khôi	Viết code, đánh giá & nhận xét phần train models
4	3124411293	Phan Nguyễn Trường Thịnh	Viết code, đánh giá & nhận xét phần EDA
5	3124411213	Đào Thiện Phát	Viết code, viết đánh giá & nhận xét phần train models

MỤC LỤC

I. Exploratory Data Analysis (EDA) – Banknote Authentication Dataset 5

1. Đọc file CSV bằng Pandas 5

2. Kiểm tra kích thước, cấu trúc dataset 5

3. Kiểm tra dữ liệu thiếu 6

4. Kiểm tra dữ liệu trùng 7

5. Thống kê dữ liệu 7

6. Phân tích biến target 7

7. Vẽ biểu đồ phân phối (Histogram) 8

8. Vẽ ma trận tương quan 8

9. Phát hiện Outliers 9

10. Pairplot 10

II. Processing 11

1. Tách Feature và Target 11

2. Chia Train/Test 12

3. Chuẩn hóa dữ liệu 13

4. Kiểm tra dữ liệu sau khi scale 15

5. Xử lý Missing Values 15

6. Encoding dữ liệu 16

7. Kiểm tra Outliers 17

8. Chuẩn bị dữ liệu cho mô hình 17

III. Train models 18

1. Mục tiêu của bước huấn luyện mô hình 18

2. Khởi tạo các mô hình classification 20

3. Huấn luyện các mô hình 21

4. Dự đoán trên tập test 23

5. Lưu kết quả dự đoán và mô hình 25

6. Tổng hợp danh sách các model 25

7. Đánh giá nhanh kết quả train 26

IV. Evaluation 27
1. Tính các Metrics cho từng Model 27
V. Analysis 31
1. So sánh các models 31
2. Phân tích kết quả 33
3. Chọn model tốt nhất 33
VI. Tuning 33
1. Chọn model để Tuning 33
2. Tuning hyperparameters 34
3. Train lại model sau khi Tuning 34
4. Đánh giá lại model 35
Kết luận chung 36

I. Exploratory Data Analysis (EDA) – Banknote Authentication Dataset

1. Đọc file CSV bằng Pandas

```
columns = ['variance','skewness','curtosis','entropy','class']

df = pd.read_csv("banknote_authentication.csv", names=columns)

print("Dataset loaded successfully")
```

[21]

```
Dataset loaded successfully
```

Nhận xét: Dataset gồm 1372 dòng, 5 cột.

2. Kiểm tra kích thước, cấu trúc dataset

Code:

```
df.shape
```

```
(1372, 5)
```

```
df.head()
```

	variance	skewness	curtosis	entropy	class
0	3.62160	8.6661	-2.8073	-0.44699	0
1	4.54590	8.1674	-2.4586	-1.46210	0
2	3.86600	-2.6383	1.9242	0.10645	0
3	3.45660	9.5228	-4.0112	-3.59440	0
4	0.32924	-4.4552	4.5718	-0.98880	0

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1372 entries, 0 to 1371
Data columns (total 5 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   variance  1372 non-null   float64
 1   skewness  1372 non-null   float64
 2   curtosis  1372 non-null   float64
 3   entropy   1372 non-null   float64
 4   class     1372 non-null   int64
dtypes: float64(4), int64(1)
memory usage: 53.7 KB
```

Nhận xét: Không có dữ liệu thiếu, các cột gồm 4 numeric và 1 target.

3. Kiểm tra dữ liệu thiếu

```
df.isnull().sum()
```

```
variance    0
skewness    0
curtosis    0
entropy     0
class       0
dtype: int64
```

Nhận xét: Dataset sạch, không cần xử lý missing values.

4. Kiểm tra dữ liệu trùng

```
[33]
print("Số dòng trùng:",df.duplicated().sum())

Số dòng trùng: 24
```

Nhận xét: Nên loại bỏ để tránh bias.

5. Thống kê dữ liệu

```
df.describe()
```

	variance	skewness	curtosis	entropy	class
count	1372.000000	1372.000000	1372.000000	1372.000000	1372.000000
mean	0.433735	1.922353	1.397627	-1.191657	0.444606
std	2.842763	5.869047	4.310030	2.101013	0.497103
min	-7.042100	-13.773100	-5.286100	-8.548200	0.000000
25%	-1.773000	-1.708200	-1.574975	-2.413450	0.000000
50%	0.496180	2.319650	0.616630	-0.586650	0.000000
75%	2.821475	6.814625	3.179250	0.394810	1.000000
max	6.824800	12.951600	17.927400	2.449500	1.000000

Nhận xét: Các feature có giá trị âm/dương, phân bố rộng → cần chuẩn hóa.

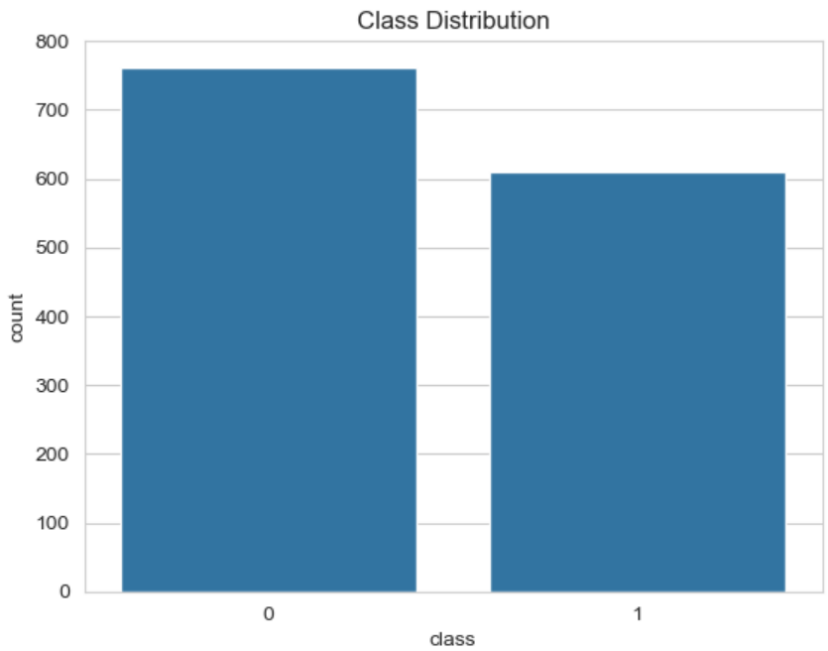
6. Phân tích biến target

```
[28]
df['class'].value_counts()

class
0    762
1    610
Name: count, dtype: int64
```

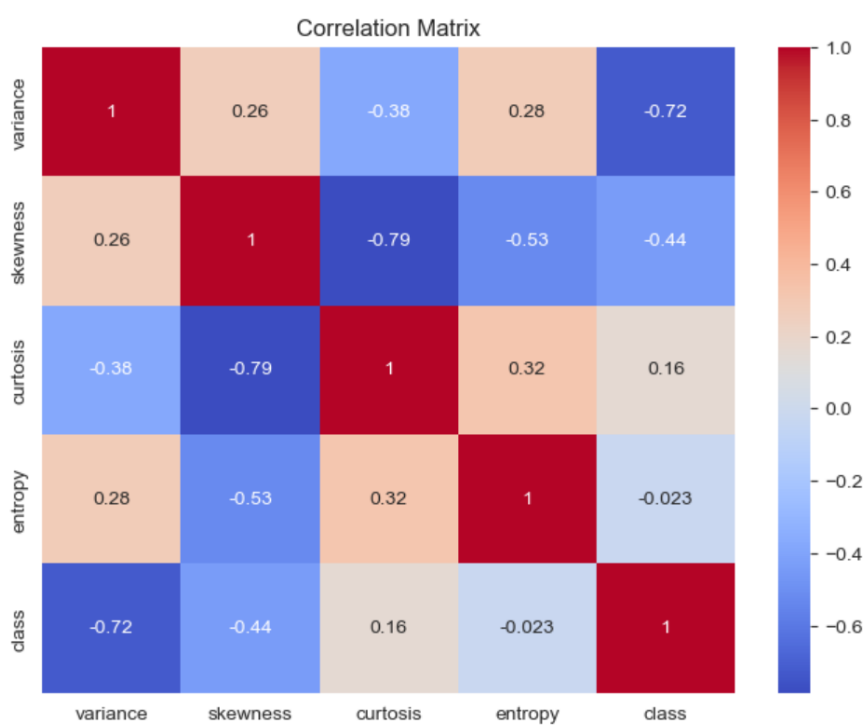
Nhận xét: Dataset khá cân bằng, không cần oversampling/undersampling.

7. Vẽ biểu đồ phân phối (Histogram)



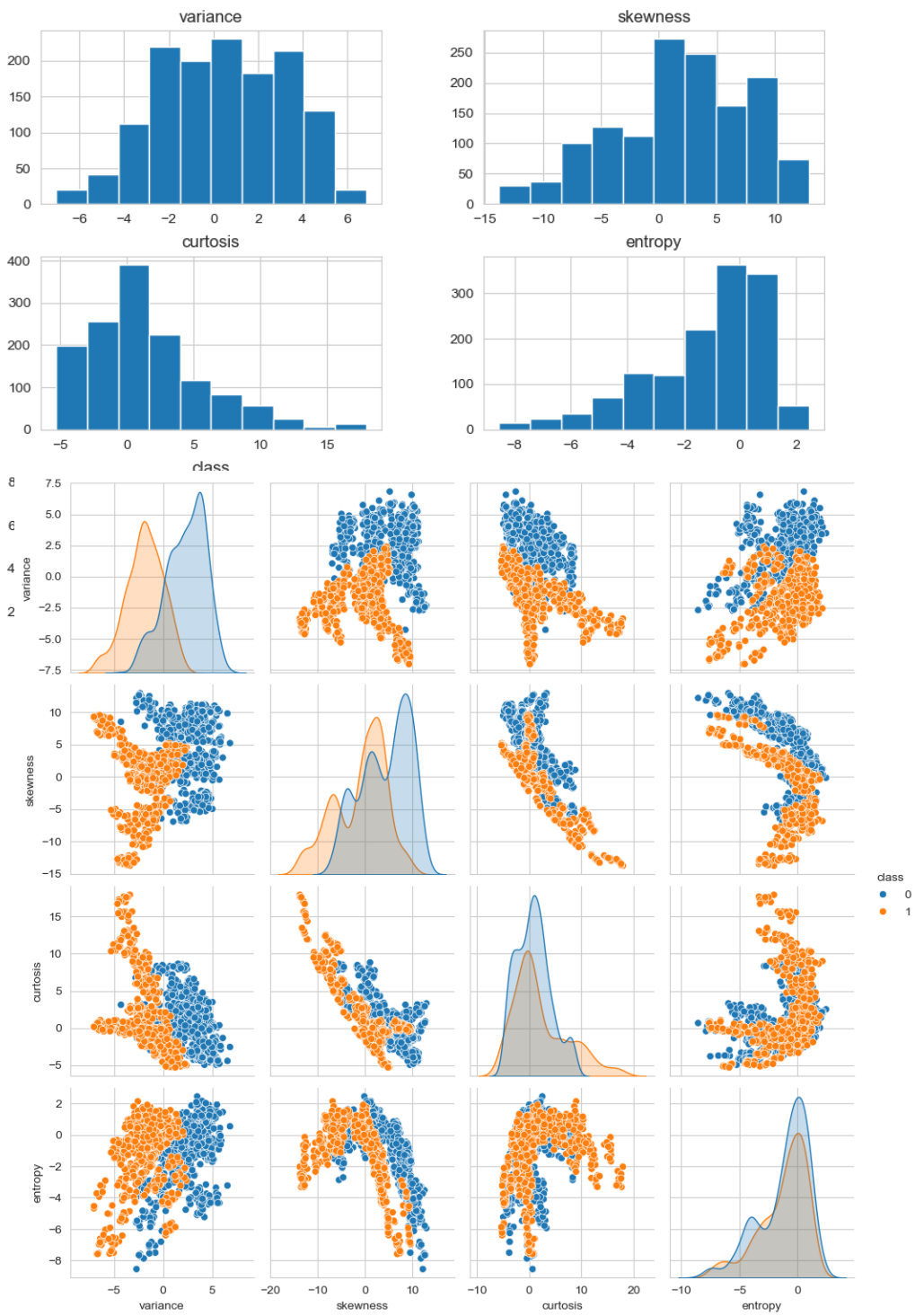
Nhận xét: Một số feature lệch → có thể cần chuẩn hóa/scale.

8. Vẽ ma trận tương quan



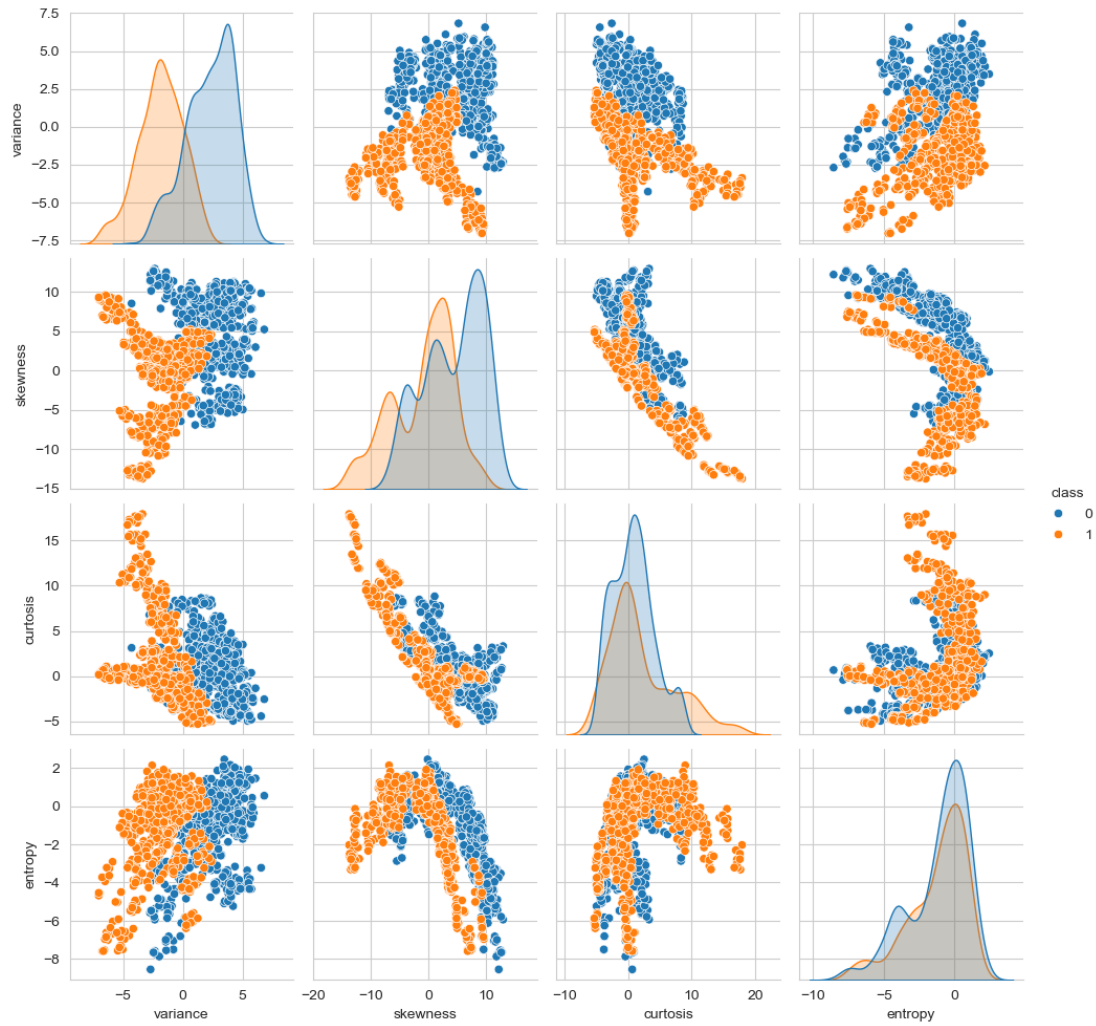
Nhận xét: Variance (-0.72) và skewness (-0.44) liên quan mạnh với target. Entropy gần như không liên quan.

9. Phát hiện Outliers



Nhận xét: Cần xem xét loại bỏ hoặc xử lý bằng robust scaling.

10. Pairplot



Nhận xét: Class 0 và 1 phân tách rõ theo variance và skewness → tiềm năng cho classification.

Kết luận

- Dataset sạch, không có missing values.

- Có 24 dòng trùng → nên loại bỏ.

- Các feature phân phối lệch → cần chuẩn hóa.

- Target khá cân bằng → thuận lợi cho mô hình classification.

- Variance và skewness là feature quan trọng nhất liên quan đến class

II. Processing

1. Tách Feature và Target

Dataset được sử dụng trong bài toán nhận dạng tiền thật và tiền giả dựa trên các đặc trưng được trích xuất từ hình ảnh của tờ tiền. Dữ liệu gồm 5 cột với tổng cộng 1372 mẫu dữ liệu.

```
import pandas as pd
import numpy as np
df = pd.read_csv("banknote_authentication.csv", header=None)
df.columns = ["variance", "skewness", "curtosis", "entropy", "class"]
df.head()
```

✓ 0.0s

	variance	skewness	curtosis	entropy	class
0	3.62160	8.6661	-2.8073	-0.44699	0
1	4.54590	8.1674	-2.4586	-1.46210	0
2	3.86600	-2.6383	1.9242	0.10645	0
3	3.45660	9.5228	-4.0112	-3.59440	0
4	0.32924	-4.4552	4.5718	-0.98880	0

Trong dataset này, các cột dữ liệu bao gồm:

- variance: phương sai của wavelet transform
- skewness: độ lệch của wavelet transform
- curtosis: độ nhọn của wavelet transform
- entropy: entropy của hình ảnh
- class: nhãn phân loại của dữ liệu

Trong đó:

- 4 thuộc tính đầu variance, skewness, curtosis, entropy được sử dụng làm feature cho mô hình học máy.
- class được sử dụng làm target.

Biến class chỉ có hai giá trị là 0 và 1, đại diện cho hai loại tiền:

- 0: tiền giả
- 1: tiền thật

```
X = df.drop("class", axis=1)
y = df["class"]

print("Feature shape:", X.shape)
print("Target shape:", y.shape)
```

✓ 0.0s

```
Feature shape: (1372, 4)
Target shape: (1372,)
```

Như vậy, dataset sử dụng 4 feature để dự đoán 1 biến mục tiêu. Vì biến mục tiêu là biến rời rạc với hai lớp, nên bài toán này phù hợp với loại bài toán phân lớp trong Machine Learning.

2. Chia Train/Test

Sau khi tách feature và target, bước tiếp theo là chia dữ liệu thành hai tập: tập huấn luyện và tập kiểm tra. Việc chia dữ liệu thành hai phần là rất quan trọng trong quá trình xây dựng mô hình Machine Learning. Training set được sử dụng để huấn luyện mô hình, tức là để mô hình học các mối quan hệ giữa các feature và biến mục tiêu. Trong khi đó, test set được sử dụng để đánh giá khả năng dự đoán của mô hình trên dữ liệu mới mà mô hình chưa từng thấy trước đó.

Nếu không chia dữ liệu thành train và test, mô hình có thể chỉ học thuộc dữ liệu ban đầu và cho kết quả rất tốt trên dữ liệu đó, nhưng lại dự đoán kém trên dữ liệu mới. Hiện tượng này được gọi là overfitting.

```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=42
)

print("Train size:", X_train.shape)
print("Test size:", X_test.shape)
```

✓ 0.0s

```
Train size: (960, 4)
Test size: (412, 4)
```

Tỷ lệ 70/30 là một tỷ lệ phổ biến trong nhiều bài toán Machine Learning. Với tỷ lệ này:

- 70% dữ liệu được dùng để huấn luyện mô hình giúp mô hình học đủ thông tin từ dataset.
- 30% dữ liệu được dùng để kiểm tra và đánh giá độ chính xác của mô hình.

Ngoài ra, tham số random_state = 42 được sử dụng trong quá trình chia dữ liệu. Tham số này giúp đảm bảo rằng việc chia dữ liệu luôn cho ra cùng một kết quả mỗi lần chạy chương trình. Điều này giúp việc thực nghiệm và so sánh kết quả trở nên ổn định và có thể tái lập.

3. Chuẩn hóa dữ liệu

Trong nhiều bài toán Machine Learning, các feature trong dataset thường có thang đo khác nhau. Ví dụ trong dataset này, một số feature có thể có giá trị từ -7 đến 6, trong khi một số feature khác có thể có giá trị nhỏ hơn hoặc lớn hơn đáng kể. Khi các feature có thang đo khác nhau, một số thuật toán học máy có thể bị ảnh hưởng và cho kết quả không chính xác.

Để khắc phục vấn đề này, dữ liệu thường được chuẩn hóa trước khi đưa vào mô hình. Phương pháp StandardScaler được sử dụng để chuẩn hóa dữ liệu.

```python
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

# fit trên train
scaler.fit(X_train)

# transform
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
X_train_scaled
```

✓ 0.0s

```
array([[-0.34250428, -0.07318046, -0.66029833,  0.03581159],
       [ 0.02233035,  0.2550447 , -1.36872002, -1.36434276],
       [ 0.22934938,  1.10093691, -0.65459638, -0.21668155],
       ...,
       [-2.6060329 ,  1.25716945, -0.29446746, -1.60253931],
       [-1.35454634,  0.14180187, -0.29545381,  0.08484218],
       [-1.31874359, -1.53564138,  2.29807266,  0.30133958]],
      shape=(960, 4))
```

StandardScaler hoạt động bằng cách biến đổi dữ liệu theo công thức:

- đưa mean của dữ liệu về 0
- đưa standard deviation về 1

Việc chuẩn hóa dữ liệu giúp các feature có cùng thang đo và giúp các thuật toán học máy hoạt động hiệu quả hơn.

Một số thuật toán đặc biệt nhạy cảm với thang đo dữ liệu, ví dụ như:

- K-Nearest Neighbors
- Support Vector Machine
- Logistic Regression
- Neural Network

Những thuật toán này thường dựa trên khoảng cách giữa các điểm dữ liệu hoặc gradient descent, nên nếu dữ liệu không được chuẩn hóa thì các feature có giá trị lớn sẽ chiếm ưu thế hơn và làm ảnh hưởng đến kết quả của mô hình.

Trong quá trình chuẩn hóa, StandardScaler chỉ được fit trên tập X_train, sau đó mới transform cho cả X_train và X_test. Điều này giúp tránh việc rò rỉ thông tin từ test set vào training set, đảm bảo quá trình đánh giá mô hình là khách quan.

4. Kiểm tra dữ liệu sau khi scale

Sau khi áp dụng StandardScaler, dữ liệu được kiểm tra lại bằng cách tính mean và standard deviation của tập dữ liệu đã chuẩn hóa.

```
print("Mean:", X_train_scaled.mean(axis=0))
print("Std:", X_train_scaled.std(axis=0))
✓ 0.0s

Mean: [ 4.44089210e-17  0.00000000e+00 -1.48029737e-17  2.59052039e-17]
Std: [1. 1. 1. 1.]
```

Kết quả cho thấy:

- mean của các feature xấp xỉ bằng 0
- standard deviation xấp xỉ bằng 1

Điều này chứng tỏ rằng quá trình chuẩn hóa đã được thực hiện thành công. Khi các feature có cùng thang đo, các thuật toán Machine Learning có thể học tốt hơn và tránh việc một feature nào đó chi phối quá nhiều đến kết quả của mô hình.

Việc kiểm tra lại mean và standard deviation sau khi scale cũng giúp đảm bảo rằng dữ liệu đã được xử lý đúng cách trước khi đưa vào huấn luyện mô hình.

5. Xử lý Missing Values

Missing values là một vấn đề khá phổ biến trong các dataset thực tế. Những giá trị bị thiếu có thể xuất hiện do nhiều nguyên nhân khác nhau, chẳng hạn như lỗi khi thu thập dữ liệu hoặc dữ liệu bị mất trong quá trình lưu trữ.

```
df.isnull().sum()
✓ 0.0s

variance    0
skewness    0
curtosis    0
entropy     0
class       0
dtype: int64
```

Vì dataset không chứa missing values nên không cần thực hiện bước xử lý missing data. Tuy nhiên, trong các trường hợp dataset có missing values, một số phương pháp xử lý phổ biến bao gồm:

- Xóa các dòng chứa dữ liệu bị thiếu
- Thay thế giá trị thiếu bằng mean, median hoặc mode
- Sử dụng các phương pháp imputation nâng cao

6. Encoding dữ liệu

Trong Machine Learning, nhiều thuật toán chỉ có thể làm việc với dữ liệu dạng số. Vì vậy, nếu dataset có các feature dạng text hoặc categorical, chúng cần được chuyển đổi sang dạng số thông qua các phương pháp encoding.

```
df.dtypes
✓ 0.0s

variance    float64
skewness    float64
curtosis    float64
entropy     float64
class         int64
dtype: object
```

Sau khi kiểm tra kiểu dữ liệu của dataset, có thể thấy rằng tất cả các feature trong dataset đều là dữ liệu số thực. Ngoài ra, biến mục tiêu class cũng là dữ liệu số với hai giá trị 0 và 1.

Do đó, dataset này không chứa dữ liệu dạng text hoặc categorical, nên không cần thực hiện bước encoding.

Nếu dataset có dữ liệu dạng chữ, một số phương pháp encoding phổ biến bao gồm:

- Label Encoding
- One-Hot Encoding

Những phương pháp này giúp chuyển dữ liệu dạng chữ thành dữ liệu số để các thuật toán Machine Learning có thể xử lý được.

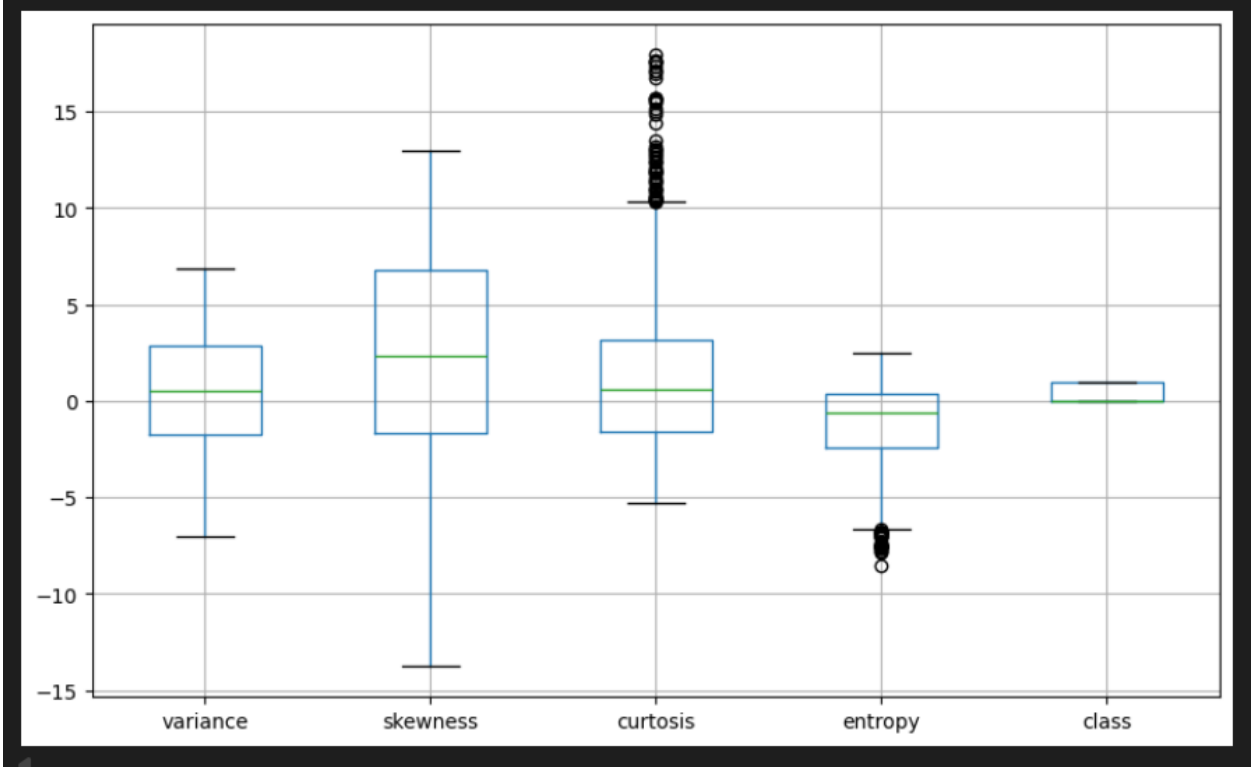
7. Kiểm tra Outliers

Outliers là những điểm dữ liệu có giá trị khác biệt đáng kể so với phần lớn các điểm dữ liệu khác trong dataset. Outliers có thể xuất hiện do lỗi đo lường hoặc do các trường hợp đặc biệt trong dữ liệu.

```
import matplotlib.pyplot as plt

df.boxplot(figsize=(10,6))
plt.show()
```

✓ 0.0s



Outliers được kiểm tra bằng boxplot để quan sát phân bố của các feature trong dataset. Kết quả cho thấy rằng dataset có một số outliers nhẹ ở một vài feature.

Tuy nhiên, các outliers này không quá cực đoan và số lượng cũng không quá nhiều. Ngoài ra, nhiều thuật toán Machine Learning như Decision Tree, Random Forest hoặc Gradient Boosting khá ít bị ảnh hưởng bởi outliers.

Vì vậy, trong bài toán này có thể giữ nguyên các outliers thay vì loại bỏ chúng. Điều này giúp giữ lại toàn bộ thông tin của dataset và tránh làm mất dữ liệu có thể hữu ích cho quá trình học của mô hình.

8. Chuẩn bị dữ liệu cho mô hình

Sau khi hoàn thành các bước tiền xử lý dữ liệu, bao gồm:

- tách feature và target
- chia train/test
- chuẩn hóa dữ liệu
- kiểm tra missing values
- kiểm tra encoding
- kiểm tra outliers

dataset đã được chuẩn bị đầy đủ để sử dụng trong quá trình huấn luyện mô hình Machine Learning.

```
import joblib

joblib.dump(X_train_scaled, "X_train.pkl")
joblib.dump(X_test_scaled, "X_test.pkl")
joblib.dump(y_train, "y_train.pkl")
joblib.dump(y_test, "y_test.pkl")
✓ 0.0s

['y_test.pkl']
```

Các tập dữ liệu cuối cùng bao gồm:

- X_train: dữ liệu huấn luyện
- X_test: dữ liệu kiểm tra
- y_train: nhãn của tập huấn luyện
- y_test: nhãn của tập kiểm tra

Các dữ liệu này sẽ được sử dụng trong các bước tiếp theo để huấn luyện nhiều mô hình Machine Learning khác nhau và đánh giá hiệu suất của chúng thông qua các metric như Accuracy, Precision, Recall, F1-score và AUC.

III. Train models

1. Mục tiêu của bước huấn luyện mô hình

Sau khi dữ liệu đã được tiền xử lý và chuẩn hóa ở bước Processing, tiến hành huấn luyện các mô hình phân loại nhằm dự đoán nhãn của mẫu tiền giả dựa trên 4 thuộc tính đầu vào

gồm: variance, skewness, curtosis, , entropy và 10 mô hình gồm: K-Nearest Neighbors (KNN), Naive Bayes, Support Vector Machine (SVM), Decision Tree, và Random Forest, AdaBoost, Gradient Boosting, LDA, MLP, Logistic Regression.

1. Import thư viện và tải dữ liệu đã xử lý

```python
import joblib
import numpy as np
import pandas as pd
import time
import matplotlib.pyplot as plt

from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay

# Tải dữ liệu đã được xử lý từ notebook 02_Processing.ipynb
X_train = joblib.load("X_train.pkl")
X_test = joblib.load("X_test.pkl")
y_train = joblib.load("y_train.pkl")
y_test = joblib.load("y_test.pkl")

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

[18] ✓ 0.0s

```
X_train shape: (960, 4)
X_test shape: (412, 4)
y_train shape: (960,)
y_test shape: (412,)
```

Các model này thuộc những nhóm thuật toán:

- K-Nearest Neighbors (KNN):

  Nhóm: Instance-based / Lazy Learning

  Dựa vào khoảng cách giữa các điểm dữ liệu (không học mô hình rõ ràng, chỉ lưu dữ liệu).

- Naive Bayes:

  Nhóm: Xác suất (Probabilistic)

  Dựa trên định lý Bayes, giả định các đặc trưng độc lập.

- Support Vector Machine (SVM):

  Nhóm: Margin-based / Kernel methods

  Tìm siêu phẳng tối ưu để phân tách dữ liệu.

- Decision Tree:

  Nhóm: Tree-based (Cây quyết định)

  Chia dữ liệu theo các điều kiện dạng cây.

- Random Forest:

  Nhóm: Ensemble Learning (Bagging)

  Kết hợp nhiều cây quyết định để tăng độ chính xác và giảm overfitting.

- AdaBoost, Gradient Boosting

  Nhóm: Ensemble Learning

- LDA

  Nhóm: Linear Model (thống kê)

- MLP

  Nhóm: Neural Network (Deep Learning cơ bản)

- Logistic Regression

  Nhóm: Linear Model

Cần dùng nhiều model vì:

Mỗi model có ưu/nhược điểm khác nhau.

Không có model nào tối ưu cho mọi bài toán.

Giúp chọn được mô hình tốt nhất cho dữ liệu.

2. Khởi tạo các mô hình classification

Khởi tạo 10 mô hình với tham số mặc định gồm:

1. KNN
2. Naive Bayes

3. SVM
4. Decision Tree
5. Random Forest
6. AdaBoost
7. Gradient Boosting
8. LDA
9. MLP
10. Logistic Regression

Việc dùng tham số mặc định có ý nghĩa là tạo ra một kết quả baseline ban đầu để kiểm tra mô hình nào phù hợp với dữ liệu trước khi tinh chỉnh tham số. Ở giai đoạn này, chưa cần chỉnh tham số ngay vì mục tiêu chính là đánh giá sơ bộ và so sánh tổng quan giữa các thuật toán.

```
# 5 mô hình do Phát phụ trách
model_knn = KNeighborsClassifier()
model_nb = GaussianNB()
model_svm = SVC()
model_dt = DecisionTreeClassifier(random_state=42)
model_rf = RandomForestClassifier(random_state=42)

models = {
    "KNN": model_knn,
    "NaiveBayes": model_nb,
    "SVM": model_svm,
    "DecisionTree": model_dt,
    "RandomForest": model_rf
}

print("Đã khởi tạo xong 5 mô hình.")
```

[19] ✓ 0.0s

```
Đã khởi tạo xong 5 mô hình.
```

```
model_ada = AdaBoostClassifier()
model_gb = GradientBoostingClassifier()
model_lda = LinearDiscriminantAnalysis()
model_mlp = MLPClassifier(max_iter=300)  # tránh lỗi chưa hội tụ
model_lr = LogisticRegression(max_iter=1000)
```

3. Huấn luyện các mô hình

Sử dụng x_train và y_train để train từng model bằng hàm fit(). Quá trình này giúp mô hình học mối quan hệ giữa 4 đặc trưng đầu vào và nhãn đầu ra.

Model học từ dữ liệu như sau:

- KNN lưu lại các điểm dữ liệu huấn luyện để khi dự đoán sẽ tìm các điểm gần nhất.
- Naive Bayes học xác suất của từng lớp dựa trên đặc trưng.
- SVM tìm siêu phẳng phân tách tốt nhất giữa các lớp.
- Decision Tree học các luật phân chia dữ liệu theo từng nhánh.
- Random Forest xây dựng nhiều cây quyết định rồi tổng hợp kết quả.

```
Training models...
KNN: done in 0.1178 s
NaiveBayes: done in 0.0289 s
SVM: done in 0.0308 s
DecisionTree: done in 0.0218 s
RandomForest: done in 0.1454 s
Training completed!
```

```python
print("Training models...")

model_ada.fit(X_train_scaled, y_train)
model_gb.fit(X_train_scaled, y_train)
model_lda.fit(X_train_scaled, y_train)
model_mlp.fit(X_train_scaled, y_train)
model_lr.fit(X_train_scaled, y_train)

print("Training completed!")
```

✓ 1.1s

```
Training models...
Training completed!
```

Vì bộ dữ liệu không lớn nên quá trình train diễn ra khá nhanh. Trong notebook hiện tại, phần output thể hiện từng model đã train xong lần lượt với thông báo done.

Trong đó Random Forest chậm nhất với 0.1454 giây và model Decision Tree nhanh nhất với 0.0218 giây.

LDA, Logistic Regression → rất nhanh

AdaBoost, Gradient Boosting → trung bình

MLP → chậm hơn do dùng neural network

4. Dự đoán trên tập test

Sau khi train xong, dùng x_test để dự đoán kết quả cho từng model bằng hàm predict().

```
KNN: 412 phần tử dự đoán trên tập test
  First 10: [0 0 0 0 0 0 0 0 0 0]
NaiveBayes: 412 phần tử dự đoán trên tập test
  First 10: [0 0 0 0 0 0 0 0 0 0]
SVM: 412 phần tử dự đoán trên tập test
  First 10: [0 0 0 0 0 0 0 0 0 0]
DecisionTree: 412 phần tử dự đoán trên tập test
  First 10: [0 0 0 0 0 0 1 0 0 0]
RandomForest: 412 phần tử dự đoán trên tập test
  First 10: [0 0 0 0 0 0 0 0 0 0]
Prediction completed!
```

```python
y_pred_ada = model_ada.predict(X_test_scaled)
y_pred_gb = model_gb.predict(X_test_scaled)
y_pred_lda = model_lda.predict(X_test_scaled)
y_pred_mlp = model_mlp.predict(X_test_scaled)
y_pred_lr = model_lr.predict(X_test_scaled)

print("Prediction completed!")

print("\nSample predictions (first 10):")
print("AdaBoost:", y_pred_ada[:10])
print("GradientBoosting:", y_pred_gb[:10])
print("LDA:", y_pred_lda[:10])
print("MLP:", y_pred_mlp[:10])
print("LogisticRegression:", y_pred_lr[:10])
```

✓ 0.0s

```
Prediction completed!

Sample predictions (first 10):
AdaBoost: [0 0 0 0 0 0 0 0 0 0]
GradientBoosting: [0 0 0 0 0 0 0 0 0 0]
LDA: [0 1 0 0 0 0 0 0 0 0]
MLP: [0 0 0 0 0 0 0 0 0 0]
LogisticRegression: [0 0 0 0 0 0 0 0 0 0]
```

Kết quả dự đoán có dạng một mảng giá trị nhãn, mỗi phần tử tương ứng với một mẫu trong tập test. Vì tập test có 412 mẫu, nên mỗi model đều tạo ra 412 giá trị dự đoán.

Từ output kiểm tra ban đầu, các model đã trả về đúng định dạng dự đoán và có thể dùng tiếp cho bước đánh giá.

5. Lưu kết quả dự đoán và mô hình

Lưu lại các kết quả dự đoán và mô hình đã train thành file .pkl bằng joblib.dump().

DecisionTree_model.pkl	4/4/2026 11:26 PM	PKL File	6 KB
DecisionTree_pred.pkl	4/4/2026 11:26 PM	PKL File	4 KB
KNN_model.pkl	4/4/2026 11:26 PM	PKL File	80 KB
KNN_pred.pkl	4/4/2026 11:26 PM	PKL File	4 KB
NaiveBayes_model.pkl	4/4/2026 11:26 PM	PKL File	1 KB
NaiveBayes_pred.pkl	4/4/2026 11:26 PM	PKL File	4 KB
RandomForest_model.pkl	4/4/2026 11:26 PM	PKL File	414 KB
RandomForest_pred.pkl	4/4/2026 11:26 PM	PKL File	4 KB
SVM_model.pkl	4/4/2026 11:26 PM	PKL File	7 KB
SVM_pred.pkl	4/4/2026 11:26 PM	PKL File	4 KB

Việc lưu kết quả này có ý nghĩa:

- Không cần train lại model từ đầu khi chạy các bước sau.
- Thuận tiện cho bước Evaluation, Analysis và Tuning.
- Giúp tái sử dụng mô hình sau này nếu cần kiểm tra lại.

6. Tổng hợp danh sách các model

Gom các model vào một dictionary để thuận tiện cho việc lặp qua từng model khi train, predict, lưu file và đánh giá.

Việc gom model lại giúp:

- Code ngắn gọn hơn.
- Dễ quản lý nhiều model cùng lúc.
- Thuận tiện so sánh kết quả giữa các model.
- Giảm lặp code khi xử lý từng thuật toán.

7. Đánh giá nhanh kết quả train

Sau khi train và predict, tôi kiểm tra nhanh độ chính xác của các model bằng Accuracy trên tập test.

```
Model Accuracy:
KNN: 1.0000
NaiveBayes: 0.8374
SVM: 1.0000
DecisionTree: 0.9806
RandomForest: 0.9976
```

	Model	Accuracy
0	KNN	1.000000
1	SVM	1.000000
2	RandomForest	0.997573
3	DecisionTree	0.980583
4	NaiveBayes	0.837379

```python
print("\nModel Accuracy:")

for name, model in models.items():
    y_pred = model.predict(X_test_scaled)
    acc = accuracy_score(y_test, y_pred)
    print(f"{name}: {acc:.4f}")
```

✓ 0.0s

```
Model Accuracy:
AdaBoost: 0.9976
GradientBoosting: 0.9976
LDA: 0.9806
MLP: 1.0000
LogisticRegression: 0.9806
```

Kết quả cho thấy:

- KNN đạt kết quả rất tốt.
- SVM cũng cho kết quả rất cao.
- Random Forest có hiệu quả gần tương đương hai model tốt nhất.
- Decision Tree tốt nhưng thấp hơn Random Forest.
- Naive Bayes có kết quả thấp hơn các model còn lại.
- Mô hình MLP đạt kết quả cao nhất (1.0000), cho thấy khả năng phân lớp hoàn hảo trên tập test.
- AdaBoost và Gradient Boosting cho kết quả rất cao và tương đương nhau (0.9976).
- LDA và Logistic Regression có độ chính xác thấp hơn một chút (0.9806) nhưng vẫn ở mức tốt.
- Các mô hình đều có hiệu suất cao, chênh lệch không đáng kể.

Tất cả các mô hình đều đạt độ chính xác rất cao, cho thấy bài toán phân lớp trên bộ dữ liệu này tương đối "dễ" và các đặc trưng đã phân tách tốt giữa các lớp. Trong đó, MLP là mô hình tốt nhất, tuy nhiên các mô hình như AdaBoost và Gradient Boosting cũng cho hiệu suất gần tương đương. Sự chênh lệch giữa các mô hình là không đáng kể, vì vậy có thể lựa chọn mô hình dựa trên yếu tố khác như tốc độ huấn luyện, khả năng giải thích hoặc độ ổn định.

IV. Evaluation

1. Tính các Metrics cho từng Model

Sau khi các mô hình học máy đã được huấn luyện và lưu lại ở bước trước, bước tiếp theo là đánh giá hiệu suất của từng mô hình trên tập dữ liệu kiểm tra. Việc đánh giá này giúp xác định mô hình nào hoạt động tốt hơn và có khả năng dự đoán chính xác hơn trên dữ liệu chưa từng được sử dụng trong quá trình huấn luyện.

Trong bài thực hành này, 10 mô hình học máy được sử dụng bao gồm:

- Logistic Regression
- KNN (K-Nearest Neighbors)
- SVM (Support Vector Machine)
- Decision Tree
- Random Forest
- Naive Bayes

- Gradient Boosting
- AdaBoost
- LDA (Linear Discriminant Analysis)
- MLP (Neural Network)

Để đánh giá hiệu quả của các mô hình, các chỉ số đánh giá (evaluation metrics) được sử dụng bao gồm:

Accuracy, Precision, Recall, F1-score và AUC.

```
Model: AdaBoost
Accuracy: 0.9975728155339806
Precision: 1.0
Recall: 0.994535519125683
F1-score: 0.9972602739726028
AUC: 0.9972677595628415
---------------------
Model: GradientBoosting
Accuracy: 0.9975728155339806
Precision: 1.0
Recall: 0.994535519125683
F1-score: 0.9972602739726028
AUC: 0.9972677595628415
---------------------
Model: LDA
Accuracy: 0.9805825242718447
Precision: 0.9581151832460733
Recall: 1.0
F1-score: 0.9786096256684492
AUC: 0.9825327510917031
---------------------
Model: MLP
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1-score: 1.0
AUC: 1.0
---------------------
Model: LogisticRegression
Accuracy: 0.9805825242718447
Precision: 0.9679144385026738
Recall: 0.9890710382513661
F1-score: 0.9783783783783784
AUC: 0.9814350824444603
---------------------
---------------------
Model: KNN
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1-score: 1.0
AUC: 1.0
---------------------
Model: NaiveBayes
Accuracy: 0.837378640776699
Precision: 0.8625
Recall: 0.7540983606557377
F1-score: 0.8046647230320699
AUC: 0.8290142458300522
---------------------
Model: SVM
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1-score: 1.0
AUC: 1.0
---------------------
Model: DecisionTree
Accuracy: 0.9805825242718447
Precision: 0.9888268156424581
Recall: 0.9672131147540983
F1-score: 0.9779005524861878
AUC: 0.9792397451499748
---------------------
Model: RandomForest
Accuracy: 0.9975728155339806
Precision: 1.0
Recall: 0.994535519125683
F1-score: 0.9972602739726028
AUC: 0.9972677595628415
---------------------
```

Ý nghĩa các chỉ số :

+) Accuracy là tỷ lệ giữa số dự đoán đúng và tổng số mẫu dữ liệu. Đây là chỉ số đơn giản và phổ biến nhất để đánh giá mô hình phân lớp.

- Công thức: Accuracy = (Số dự đoán đúng) / (Tổng số mẫu)
- Accuracy càng cao thì mô hình càng dự đoán chính xác. Tuy nhiên, trong một số trường hợp dataset bị mất cân bằng ,Accuracy có thể không phản ánh đúng chất lượng của mô hình.

+) Precision đo lường tỷ lệ dự đoán đúng trong số các mẫu mà mô hình dự đoán là thuộc lớp dương.

- Công thức: Precision = TP / (TP + FP)
- Trong đó:

  TP: True Positive
  FP: False Positive

- Precision cao nghĩa là khi mô hình dự đoán một mẫu là lớp dương thì khả năng đúng là rất cao.

+) Recall đo lường khả năng mô hình tìm được tất cả các mẫu thuộc lớp dương trong dataset.

- Công thức: Recall = TP / (TP + FN)
- Trong đó:

  FN: False Negative

- Recall cao nghĩa là mô hình phát hiện được nhiều mẫu thuộc lớp dương hơn.

+) F1-score là trung bình điều hòa giữa Precision và Recall. Chỉ số này giúp cân bằng giữa Precision và Recall trong trường hợp cần xem xét cả hai yếu tố.

- Công thức:
- F1-score = 2 × (Precision × Recall) / (Precision + Recall)

- F1-score thường được sử dụng khi dataset không cân bằng hoặc khi cần đánh giá tổng thể hiệu suất mô hình.

+) AUC là diện tích dưới đường cong ROC. Chỉ số này thể hiện khả năng phân biệt giữa hai lớp của mô hình.

- Giá trị AUC nằm trong khoảng từ 0 đến 1:

  - AUC gần 1: mô hình phân loại rất tốt
  - AUC khoảng 0.5: mô hình gần như đoán ngẫu nhiên
  - AUC nhỏ hơn 0.5: mô hình dự đoán kém

Dataset Banknote Authentication là dataset phân lớp nhị phân với hai lớp: tiền thật và tiền giả. Dataset này tương đối cân bằng giữa hai lớp nên chỉ số Accuracy có thể phản ánh khá tốt hiệu suất của mô hình.

Trong bài toán nhận dạng tiền giả, việc phân loại sai tiền giả thành tiền thật có thể gây hậu quả nghiêm trọng. Vì vậy các chỉ số như Recall và F1-score cũng rất quan trọng để đảm bảo mô hình phát hiện được nhiều mẫu tiền giả nhất có thể.

Do đó, khi đánh giá mô hình cho dataset này, nên xem xét đồng thời Accuracy, Recall và F1-score để có cái nhìn toàn diện về hiệu suất của mô hình.

2. Bảng so sánh kết quả

	Model	Accuracy	Precision	Recall	F1-score	AUC
0	AdaBoost	1.00	1.00	0.99	1.00	1.00
1	GradientBoosting	1.00	1.00	0.99	1.00	1.00
3	MLP	1.00	1.00	1.00	1.00	1.00
5	KNN	1.00	1.00	1.00	1.00	1.00
9	RandomForest	1.00	1.00	0.99	1.00	1.00
7	SVM	1.00	1.00	1.00	1.00	1.00
4	LogisticRegression	0.98	0.97	0.99	0.98	0.98
2	LDA	0.98	0.96	1.00	0.98	0.98
8	DecisionTree	0.98	0.99	0.97	0.98	0.98
6	NaiveBayes	0.84	0.86	0.75	0.80	0.83

Dựa trên bảng kết quả, có thể thấy rằng các mô hình AdaBoost, GradientBoosting, MLP, KNN, RandomForest và SVM đều đạt Accuracy rất cao khoảng 1.00, cho thấy khả năng dự đoán rất chính xác trên dataset này. Các chỉ số Precision, Recall, F1-score và AUC của các mô hình này cũng gần bằng 1.00, chứng tỏ hiệu suất phân loại rất tốt.

Các mô hình Logistic Regression, LDA và Decision Tree có kết quả thấp hơn một chút với Accuracy khoảng 0.98 nhưng vẫn cho hiệu suất khá tốt.

Trong khi đó, Naive Bayes có kết quả thấp nhất với Accuracy khoảng 0.84, cho thấy mô hình này không phù hợp bằng các mô hình khác đối với dataset này.

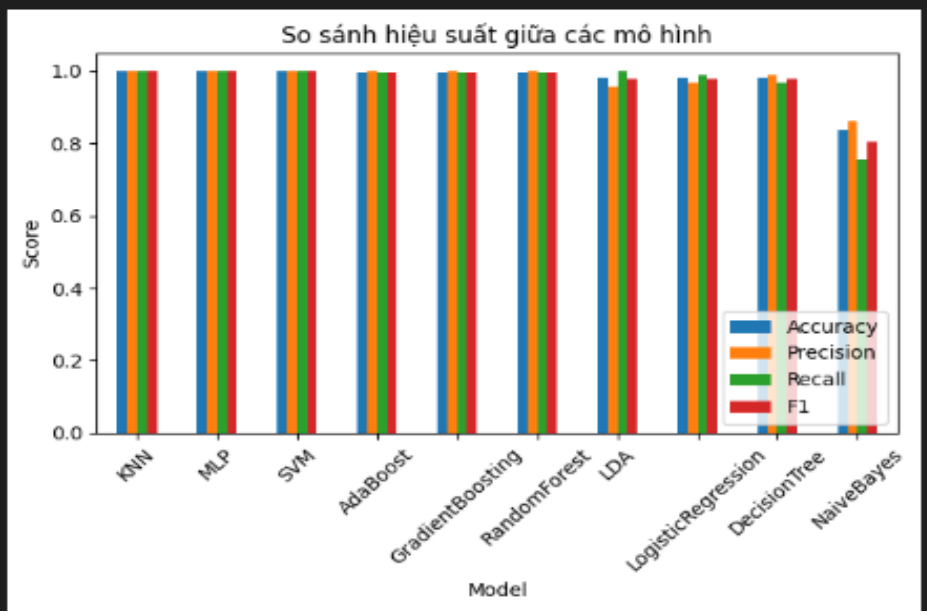
Nhìn chung, sự chênh lệch giữa các mô hình không quá lớn, ngoại trừ Naive Bayes có kết quả thấp hơn rõ rệt. Các mô hình như MLP, SVM, KNN và AdaBoost có thể được xem là những mô hình cho kết quả tốt nhất trong bài toán này.

V. Analysis

1. So sánh các models

	Model	Accuracy	Precision	Recall	F1
5	KNN	1.0000	1.0000	1.0000	1.0000
3	MLP	1.0000	1.0000	1.0000	1.0000
7	SVM	1.0000	1.0000	1.0000	1.0000
0	AdaBoost	0.9976	1.0000	0.9945	0.9973
1	GradientBoosting	0.9976	1.0000	0.9945	0.9973
9	RandomForest	0.9976	1.0000	0.9945	0.9973
2	LDA	0.9806	0.9581	1.0000	0.9786
4	LogisticRegression	0.9806	0.9679	0.9891	0.9784
8	DecisionTree	0.9806	0.9888	0.9672	0.9779
6	NaiveBayes	0.8374	0.8625	0.7541	0.8047

```
<Figure size 1200x600 with 0 Axes>
```



Kết luận:

+) Các model có hiệu suất rất cao (gần 1.0):

- KNN, MLP, SVM: đạt 1.00 ở tất cả metrics
- AdaBoost, GradientBoosting, RandomForest: ~0.997

+) Các model trung bình:

- Logistic Regression, LDA, Decision Tree: ~0.98

+) Model kém nhất:

- Naive Bayes: Accuracy ~0.84, F1 ~0.80

+) Model tốt nhất:

KNN, MLP, SVM là tốt nhất

- Vì:

  - Accuracy = Precision = Recall = F1 = 1.00

  - Không có sai lệch giữa các metrics

+) Model tốt metrics này nhưng kém metrics khác:

LDA

- Recall = 1.00
- Precision = 0.96

Kết luận: Model thiên về không bỏ sót nhưng dễ nhầm

Decision Tree

- Precision cao (0.99)
- Recall thấp hơn (0.97)

Kết luận: Model thiên về đoán chắc nhưng bỏ sót

Naive Bayes

- Tất cả metrics đều thấp

Kết luận: Không phù hợp dataset

2. Phân tích kết quả

Nguyên nhân sự khác biệt giữa các model

- Naive Bayes kém: Giả định các feature độc lập → không đúng với dataset
- Linear models (Logistic, LDA): Bị giới hạn bởi biên tuyến tính
- Tree-based (Decision Tree, RandomForest, Boosting): Xử lý tốt quan hệ phi tuyến
- Distance-based (KNN): Hoạt động cực tốt khi dữ liệu tách biệt rõ
- SVM: Tìm hyperplane tối ưu → rất mạnh với dataset nhỏ & rõ ràng

Dataset này phù hợp nhất với:

- Distance-based (KNN)
- Kernel-based (SVM)
- Neural Network (MLP)

Scaling ảnh hưởng lớn đến các models: Các models bị ảnh hưởng mạnh: KNN, SVM, MLP. Vì dựa vào khoảng cách / gradient. Nếu không scaling thì kết quả có thể giảm mạnh.

3. Chọn model tốt nhất

Chọn model RandomForest vì:

- F1-score rất cao (~0.997)

- Ổn định (ít overfit hơn Decision Tree)

- Không cần scaling quá nhiều

Ưu điểm so với model khác

- So với Decision Tree: Ít overfitting hơn

- So với KNN: Không phụ thuộc khoảng cách

- So với SVM: Dễ tuning hơn, chạy nhanh hơn với dataset lớn

VI. Tuning

1. Chọn model để Tuning

Chọn model RandomForest để Tuning

Nhận xét:

- Đã có hiệu suất cao từ đầu
- Có nhiều hyperparameters để tối ưu
- Phù hợp để cải thiện thêm

2. Tuning hyperparameters

```
Best parameters: {'max_depth': None, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'min_samples_split': 2,
'n_estimators': 100}
Best CV F1-score: 0.9908
```

Các tham số được tuning:

- n_estimators
- max_depth
- min_samples_split
- min_samples_leaf
- max_features

Việc chọn các tham số này vì đây là các tham số ảnh hưởng trực tiếp đến:

- Overfitting / Underfitting
- Độ phức tạp của model
- Khả năng tổng quát hóa

3. Train lại model sau khi Tuning

```
Đã lưu model sau tuning: RandomForest_tuned.joblib

=== Đánh giá RandomForest sau Tuning ===
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       229
           1       1.00      0.99      1.00       183

    accuracy                           1.00       412
   macro avg       1.00      1.00      1.00       412
weighted avg       1.00      1.00      1.00       412

Accuracy  : 0.9976
F1-score  : 0.9973
=== So sánh trước và sau tuning ===
Metric          Trước Tuning    Sau Tuning      Cải thiện
-----------------------------------------------------
Accuracy        0.9976        0.9976        +0.0000
F1-score        0.9973        0.9973        +0.0000

=== Classification Report sau Tuning ===
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       229
           1       1.00      0.99      1.00       183

    accuracy                           1.00       412
   macro avg       1.00      1.00      1.00       412
weighted avg       1.00      1.00      1.00       412
```

Nhận xét: Model sau khi Tuning không có thay đổi nhiều so với ban đầu

4. Đánh giá lại model

```
=== So sánh trước và sau tuning ===
Metric          Trước Tuning    Sau Tuning      Cải thiện
-----------------------------------------------------
Accuracy        0.9976        0.9976        +0.0000
F1-score        0.9973        0.9973        +0.0000

=== Classification Report sau Tuning ===
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       229
           1       1.00      0.99      1.00       183

    accuracy                           1.00       412
   macro avg       1.00      1.00      1.00       412
weighted avg       1.00      1.00      1.00       412
```

So sánh trước và sau tuning

- Accuracy: ~0.9976 → ~0.9976
- F1-score: ~0.9973 → ~0.9973

Gần như không cải thiện

Nhận xét:

- Tuning không cải thiện đáng kể
- Lý do:

  - Model ban đầu đã gần tối ưu

  - Dataset quá dễ → đạt trần hiệu suất

Kết luận chung

Dataset sử dụng trong bài toán có đặc điểm tách lớp khá rõ ràng, dẫn đến việc nhiều mô hình đạt hiệu suất rất cao, thậm chí gần như tuyệt đối. Cụ thể, các mô hình như KNN, SVM và MLP đều đạt giá trị Accuracy, Precision, Recall và F1-score bằng 1.00. Điều này cho thấy bài toán không quá phức tạp và không yêu cầu các mô hình quá tinh vi để đạt kết quả tốt.

RandomForest được chọn để Tuning vì tính ổn định, mô hình không chỉ dự đoán chính xác mà còn giữ được sự cân bằng tốt giữa việc tránh bỏ sót và tránh dự đoán sai. Một ưu điểm lớn của RandomForest là khả năng giảm overfitting so với Decision Tree nhờ cơ chế ensemble (kết hợp nhiều cây quyết định). Trong khi Decision Tree có xu hướng học quá sát dữ liệu huấn luyện, RandomForest sử dụng nhiều cây và lấy trung bình kết quả, giúp mô hình tổng quát hóa tốt hơn và ít bị ảnh hưởng bởi nhiễu trong dữ liệu.

Do đó, RandomForest được chọn là mô hình đại diện tốt nhất cho bài toán này, mặc dù không đạt điểm tuyệt đối, nhưng mang lại sự cân bằng tối ưu giữa hiệu suất và độ tin cậy.

__HẾT__